

Retrieval on Receipts

How every food and drink expense became a searchable vector, and how that vector store started answering the question the price tables never could.

497	expenses scanned	2048	dimensions per vector
180	indexed as vectors	1	new dependency (numpy)
27	drink / 153 food	0.8.0	pgvector, on Neon

September 2026. Every figure measured against the live database, not estimated.

WHY The tables know prices. They don't know you.

The recommender already stood on two solid tables: 1,254 liquor prices across five states, most of them parsed from the states' own PDFs, and 66 Delhi NCR restaurants with cited costs for two. Both are public, static and correct.

Neither knows that you and Anubhav drank Old Monk in March and paid Rs 675. That second thing is the half a generic recommender can never have - and until now the only route to it was a hand-written regex, roughly forty lines of alternation that had already been patched three times for exactly the mistakes a person would never make:

- 'gin' matching Monginis, a bakery, putting cake in the liquor data
- 'social' matching an unrelated expense and inventing a favourite restaurant
- 'Export' as a channel marker versus Tuborg Mild Export Beer, an actual beer

Each fix was correct and each was reactive. A regex encodes the failures you have already seen. The point of this work is a representation that generalises to the ones you haven't.

MAP The shape of the whole thing

Seven stages. The first four run when an expense is saved; the last three run when a recommendation is asked for.

```
EXPENSE SAVED
|
+--> 01 CLASSIFY    drink, food, or neither
+--> 02 FEATURISE   words, pairs, 4-char shingles
+--> 03 EMBED       hash -> 2048 floats, unit length
+--> 04 LINK        nearest catalogue entry, cosine
+--> 05 STORE       pgvector column, one row per expense

RECOMMENDATION ASKED
|
+--> 06 RETRIEVE    cosine, scoped to your groups
+--> 07 RECOMMEND   your spend, inside tonight's budget
```

One new dependency across the whole build: numpy. No model is downloaded, nothing is sent to an API, and nothing needs a GPU - which is the only reason it fits inside a 512 MB free tier that also spins down when idle.

01 Deciding what is even food

The regexes did not disappear - they were promoted. All three now live in one module and are imported by both routers and the indexer, because the indexer and the recommender agreeing on the answer matters more than either answer being clever.

Classification is deliberately coarse: drink, food, or nothing. Groceries are excluded outright - half the 'snacks' in your data are a Zepto run or raw chicken, which say a great deal about your kitchen and nothing about which restaurant to pick.

BUCKET	ROWS	WHAT IT IS
drink	27	Bottles, rounds, bar tabs
food	153	Meals out, deliveries, snacks
skipped	317	Fuel, rent, cabs, groceries

Two thirds of your expenses are correctly ignored. An index that swallowed everything would retrieve confidently and uselessly.

02 Turning a scrap of text into features

Expense titles are not sentences. They are things like 'newbrandtry(half)+momos+chips' and 'oldmonk 750'. Three feature families, chosen for that reality:

```
"Old Monk 750ml Snacks"

normalise -> old monk 750 ml snacks

words      w:old w:monk w:750 w:ml w:snacks
pairs      b:old_monk b:monk_750 b:750_ml
shingles    c:^old c:old$ c:^mon c:monk c:onk$ ...
```

The four-character shingles are what make this survive contact with real typing. 'oldmonk', 'old monk' and a misspelling share most of their shingles even when they share no whole word.

ONE LINE THAT MATTERED MORE THAN ANY OTHER

Normalisation splits letters from digits: vat69 -> vat 69, 750ml -> 750 ml. People type it closed up; the catalogue prints it open. Without that split the true match vat69 -> VAT 69 BLENDED SCOTCH WHISKY scored 0.17, which was below several false matches. With it, 0.30 before any other change.

03 From features to a vector

Each feature is hashed to a dimension, weighted by how often it occurs, and the whole vector is scaled to unit length.

```
d      = crc32(feature) mod 2048
v[d]   = 1 + ln(count)
v      = v / ||v||
```

- crc32, not Python's hash(). String hashing is randomised per process, so hash() would produce a different vector after every restart and silently poison everything already stored.
- $1 + \ln(\text{count})$ is sublinear: a word appearing ten times is more important than once, but not ten times more.
- Unit length means cosine similarity is just the dot product - no division at query time, and long titles stop beating short ones on length alone.

This is the classic hashing trick. It has one real cost, which turned out to be the whole story of the next stage: two different features can land on the same dimension, and the vector cannot tell them apart.

04 Choosing 2048, by measurement

I first picked 384 dimensions for a bad reason: it is the width of all-MiniLM-L6-v2, so a neural model could be swapped in later without migrating the column. Forward-compatibility is a real consideration. It is not a reason to ship a broken retriever.

WHAT 384 DIMENSIONS ACTUALLY DID

'momos' matched Karim's at 0.338, while the true 'vat69' -> VAT 69 BLENDED SCOTCH WHISKY scored 0.172. The false pair outranked the true one. There is no threshold that rescues that - any cut-off admitting the truth also admits the nonsense.

So I built a set of true and false pairs and measured. The margin is the gap between the worst true match and the best false one; below zero means no threshold can separate them.

DIMS	TRUE MIN	FALSE MAX	MARGIN	PER ROW	VERDICT
384	0.304	0.338	-0.034	1.5 KB	Unusable
1536	0.304	0.169	+0.135	6 KB	Workable
2048	0.304	0.075	+0.229	8 KB	Chosen
4096	0.304	0.075	+0.229	16 KB	No gain, 2x cost

2048 is exactly where separation stops improving. 4096 buys nothing and costs double. The match threshold sits at 0.25, comfortably inside a gap running from 0.075 to 0.304.

At 8 KB a row that is about 1.4 MB today and roughly 40 MB at five thousand expenses - against a 500 MB free tier currently 1.6% used. Storage is not the constraint and won't be for years.

05 Why lexical, and where the seam is

These are lexical embeddings, not neural ones. That deserves saying plainly, because 'embedding' has come to imply a transformer.

Two reasons, one practical and one principled. The practical one: Render's free tier is 512 MB and sentence-transformers pulls in torch at roughly 800 MB. It does not fit, and there is no API key in play.

The principled one matters more. Matching 'old monk 750' to OLD MONK THE ORIGINAL PREMIUM STRONG BEER is a lexical problem. The words genuinely are the same words, just buried in a registered label. Character shingles handle the abbreviations and misspellings people type into an expense box. A neural model would spend a great deal of compute rediscovering that the string 'old monk' appears in both.

QUERY	CATALOGUE ENTRY	COSINE
old monk	OLD MONK THE ORIGINAL ... BEER	0.465
oldmonk 750	Old Monk	0.426
vat69	VAT 69 BLENDED SCOTCH WHISKY	0.430
chinese dinner	Asia Kitchen	0.075
momos	Karim's	0.000

What this genuinely cannot do is semantics. It will never learn that 'nightcap' means whisky, or that a Riesling and a Chardonnay are neighbours. When that becomes worth paying for, embed() is the single function to replace - everything downstream takes a list of floats and does not care where they came from.

06 Linking, and letting retrieval overrule the rules

Every catalogue entry - 1,254 bottles and 66 restaurants - is embedded once and cached. A new expense is compared against all of them, and the nearest entry above 0.25 becomes its canonical label. That is what collapses 'old monk', 'Old monk Drinks' and 'oldmonk 750' into a single suggestion instead of three.

Then the real data produced something better than the design.

FOUND IN YOUR EXPENSES

'Beer Cafe' is a restaurant in Cyber Hub. But DRINK_RE sees the word beer and claims it first, so it was classified as a drink and dutifully linked to Stok Lager Beer - a plausible, confident, wrong answer that would have appeared as something you drink.

THE FIX

Retrieval is now allowed to overrule the regex. Each expense is scored against both catalogues, and if the other side wins by a clear margin - 0.10, not a hair - the expense is refiled. 'Beer Cafe' now classifies as food and links to The Beer Cafe. The margin matters: a small edge either way is noise at these scores, and a category the regex was confident about should not flip on noise.

A second flaw showed up in the same pass. Expenses with nothing distinctive in them - 'Food expense Food', 'snacks' - were becoming suggestions that literally read 'food expense food'. Category words are now stripped from any fallback label, and an entry with nothing left over gets no label at all, which keeps it out of the results entirely. A suggestion you cannot act on is worse than one fewer suggestion.

07 Storage: pgvector, and no index

Vectors live in a vector(2048) column on Neon, which already had pgvector 0.8.0 available. Cosine distance is computed by the database with the <=> operator, so ranking never travels over the wire.

```
SELECT label, amount, occurred_on,
       1 - (embedding <=> CAST(:q AS vector)) AS score
FROM   knowledge_items
WHERE  kind = :kind
      AND group_id = ANY(:gids)      -- the whole security model
ORDER BY embedding <=> CAST(:q AS vector)
LIMIT  :lim
```

There is deliberately no vector index. At 180 rows an IVFFlat or HNSW index would be ignored by the planner and would cost accuracy for nothing; exact search over a few hundred rows is already sub-millisecond. That decision is worth revisiting somewhere north of a hundred thousand rows, which on your current pace is a long way off.

No pgvector Python package either. The extension accepts its own text form, so the vector is written with a cast - the only raw SQL in the codebase, and contained to one file.

08 Staying live

The knowledge base would be worth much less if it needed rebuilding by hand. Indexing hangs off the expense write path, inside the same transaction, so an expense and its vector can never disagree.

- Create - indexed before commit. Tonight's drinks are retrievable before the next recommendation is asked for.
- Edit - re-indexed. An edit can change the amount, the date, or whether the thing is food at all.
- Delete - the row goes with it, by foreign key cascade.
- Reclassify - an expense that stops looking like food is removed, not left as a stale vector still answering the old question.

A DELIBERATE SWALLOW

Indexing failures are caught and logged, never raised. A knowledge-base problem must never be the reason somebody cannot record what they spent. The cost of failure is one row in a retrieval index that a reindex repairs; the cost of raising is a person's expense.

One row per expense, enforced by a unique constraint, so re-indexing updates rather than piling up. The full rebuild is idempotent and admin-only.

09 Retrieval becomes a recommendation

This is the part that closes the loop. Indexed history no longer only ranks the published tables - it produces suggestions of its own. Things you have actually bought, grouped by canonical label, whose typical spend falls inside the budget you set:

Uttar Pradesh . 2 people . Rs 300-1200 . with Anubhav

Old Monk	x5	avg Rs 541	last 2026-03-15
Smirnoff	x3	avg Rs 620	last 2026-06-19
Budweiser Magnum Beer	x3	avg Rs 483	last 2026-04-20
Black Dog	x2	avg Rs 470	last 2026-08-20

ONE WORD DOING A LOT OF WORK

The figure is spend, not a price. A single expense can be a round for six or one bottle, and nothing in the data can tell which. So the API calls it spend, the page calls it spend, and neither implies a shelf price it cannot support. This is the same discipline as the ~ in front of a restaurant estimate.

The effect is that a bottle nobody published a price for - a local label, a one-off, something from a shop that only you go to - becomes recommendable purely because you bought it and it fits tonight's budget.

10 Who can see what

The index spans every expense in the app, across everybody's groups. Search is therefore scoped exactly the way every other read here is - to groups the caller belongs to. An unscoped vector search would be a very efficient way to read what other people drink.

CALLER	HITS	WHY
Anukul	5	In the groups those expenses live in
Utkarsh	0	Shares none of those groups
unscoped	0	An empty group list returns nothing, by construction

The group id is denormalised onto every row precisely so this filter needs no join. The cheapest possible check is the one most likely to still be there in a year.

END What this is not, and what comes next

HONEST LIMITS

- No semantics. Lexical vectors will not connect 'nightcap' to whisky. Replacing embed() is the upgrade path.
- No generation. Nothing here asks a language model. Retrieval picks rows; arithmetic is done in code. Every number on screen traces to a row or a receipt.
- The classifier is still regex. Retrieval can now overrule it on category, but the initial gate is unchanged. Embedding-based classification is the obvious next step.
- The backfill is slow: 176 seconds for 180 rows, one round trip to Neon per row. Fine as a one-off, and a batched write if it is ever run often.

WHAT WOULD ACTUALLY EARN ITS KEEP NEXT

Not a bigger model. The highest-value thing left is a feedback loop this app is unusually well placed to close: it already records what people spend. Log which suggestions were shown, join that against the expenses that follow, and you learn whether the advice was taken - ground truth that most recommenders never see.

That is a table and a join. It needs no vectors at all, and it is what would make the system genuinely learn rather than merely remember.

SplitEasy - knowledge base built over 497 expenses, 180 indexed, 2048 dimensions, pgvector 0.8.0 on Neon. All figures measured against the live database, not estimated. Frontend changes in this work are unverified: there is no Node toolchain on the build machine, so the pages were checked by bracket-balance against their committed versions rather than compiled.